

**T.C.
KASTAMONU ÜNİVERSİTESİ
MÜHENDİSLİK VE MİMARLIK FAKÜLTESİ
BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ**



NESNEYE YÖNELİK PROGRAMLAMA

PROJE RAPORU

Proje Sahibi	Gizem KARAASLAN
Okul No	214410036
Danışman	Ahmet Nusret ÖZALP
Ders Kodu	BSM204
Tarih	30.05.2026

API Çağrı İzleme ve Tehdit Tespiti Sistemi: C++17'de Polimorfik, Kural Tabanlı Davranışsal Analiz Motoru

Özet—Bu çalışmada, nesneye yönelik programlamanın (NYP) dört temel ilkesi — soyutlama, kalıtım, polimorfizm ve kapsülleme — kullanılarak C++17'de geliştirilen davranışsal bir API çağrı izleme ve tehdit tespiti sisteminin tasarım ve uygulaması sunulmaktadır. Sistem, simüle edilmiş Windows API çağrı dizilerini ele geçirmekte, bunları takılabilir tespit kuralları kümesine karşı değerlendirmekte ve gerçek zamanlı olarak şiddet etiketli uyarılar üretmektedir. Üç somut kural sınıfı — RansomwareRule, CodeInjectionRule ve SuspiciousNetworkRule — saf sanal bir taban sınıftan (DetectionRule) türetilmiş ve hepsini polimorfik olarak bir API çağrı geçmişine uygulayan DetectionEngine tarafından düzenlenmektedir. Dört simülasyon senaryosu — fidye yazılımı şifreleme, DLL enjeksiyonu, veri sızıntısı ve zararsız kontrol senaryosu — tespit mantığını doğrulamaktadır.

Anahtar Kelimeler—davranışsal tespit, API izleme, nesneye yönelik programlama, polimorfizm, kalıtım, soyutlama, kapsülleme, fidye yazılımı tespiti, kod enjeksiyonu, veri sızıntısı, C++17, tehdit tespiti

I. GİRİŞ

Sofistike kötü amaçlı yazılımların çoğunluğunun yayılması, statik imza tabanlı antivirüs tespitini giderek yetersiz kılmaktadır. Fidye yazılımları, süreç enjeksiyon araç setleri ve komuta-kontrol (K2) implantları dâhil olmak üzere modern tehditler; bayt kalıplarını algılamaktan kaçınmak için rutin olarak gizlenmekte, paketlenmekte veya polimorfik olarak mutasyona uğratılmaktadır. Bir programın neye benzediğinden ziyade ne yaptığına odaklanan davranışsal tespit, bu nedenle çağdaş uç nokta koruma platformlarının köşe taşı hâline gelmiştir [1].

İşletim sistemi düzeyinde kötü niyetli davranışın gözlemlenebilir birimi Windows API çağrısıdır. Fidyeye yazılımları; şifreleme soneki uzantılı yollara yapılan yoğun WriteFile çağrı patlamaları ve vssadmin ya da wmic aracılığıyla Gölge Kopya silme işlemiyle karakterize edilmektedir. DLL enjeksiyonu, kurallasalmış üç API dizisini izlemektedir: OpenProcess, VirtualAllocEx ve WriteProcessMemory veya CreateRemoteThread. Veri sızıntısı; hassas yollardaki ReadFile işlemleriyle 4444 veya 1337 gibi bilinen K2 portlarında giden ağ aktivitesini birleştirmektedir [2].

Bu çalışma, bu davranışsal kalıpları ortak soyut bir taban sınıftan türetilmiş takılabilir kural nesneleri olarak modelleyen bağımsız bir C++17 sistemi açıklamaktadır. Motor, süreç başına API çağrı geçmişini tutmakta ve her yeni çağrıyı kayıtlı tüm kurallara karşı değerlendirmektedir. Dört NYP ilkesi açıkça uygulanmaktadır: saf sanal DetectionRule arayüzü aracılığıyla soyutlama, üç somut kural alt sınıfı aracılığıyla kalıtım, DetectionEngine::processCall() içinde sanal yönlendirme aracılığıyla polimorfizm ve DetectionEngine ile DetectionRule'un her ikisinde de özel üye erişim denetimi aracılığıyla kapsülleme.

II. İLGİLİ ÇALIŞMALAR

A. Davranışsal Kötü Amaçlı Yazılım Tespiti

Kötü amaçlı yazılımın davranışsal analizi, 2000'lerin başında dinamik analiz sanal alanlarının girişinden bu yana kapsamlı biçimde araştırılmıştır [3]. Cuckoo Sandbox [4] ve TTAlyze [5] gibi sistemler, kötü amaçlı yazılım örneklerini izole sanal makinelerde yürütmekte ve API çağrı dizilerini, ağ trafiğini ve dosya sistemi değişikliklerini kaydetmektedir.

API çağrısı n-gram modelleri, kötü amaçlı yazılım ailesi tanımlaması için güçlü sınıflandırma doğruluğu göstermiştir [6]. Daha yeni çalışmalar ise API çağrı dizilerine yinelemeli sinir ağlarını ve transformatör mimarilerini uygulamıştır [7]. Bu araç, öğrenilmiş modellere göre daha az genel olmakla birlikte tamamen denetlenebilir ve eğitim verisi gerektirmeyen yorumlanabilir kural tabanlı yaklaşımı benimsemektedir.

B. Güvenlik Yazılımında Nesneye Yönelik Tasarım

Güvenlik araçları tarihsel olarak performans ve düşük seviyeli erişim gereksinimleri nedeniyle C ile yazılmıştır. C++'ın güvenlik araçlarında benimsenmesi, yüksek performansı korurken artan kod karmaşıklığını yönetme gereksinimiyle yönlendirilmiştir. Bu çalışmada DetectionRule hiyerarşisi aracılığıyla gerçekleştirilen Strateji ve Şablon Yöntemi tasarım kalıpları, genişletilebilir güvenlik politikası motorları için yaygın olarak önerilmektedir [8].

C. API Çağrısı İzleme

Kullanıcı modu API çağrısı izleme, satırıçi kancalar, içeri aktarma adres tablosu (IAT) yaması ve çekirdek geri arama kaydı aracılığıyla üretim güvenlik ürünlerinde uygulanmaktadır [9]. Eğitim amacıyla bu araç, önceden oluşturulmuş APICall nesnelerini kabul ederek izleme katmanını simüle etmekte; davranışsal mantığın çekirdek modu erişimi veya platforma özgü kancalama altyapısı gerektirmeksizin incelenmesine olanak tanımaktadır.

III. VERİ YAPILARI

A. APICall Yapısı

APICall yapısı, tek bir gözlemlenmiş API çağrısıyla ilişkili verileri kapsüllemektedir. Başlatan sürecin adını (processName), çağrılmış Windows API fonksiyonunun adını (apiName), bir dizi bağımsız değişken (args) ve ISO 8601 zaman damgasını (timestamp) depolamaktadır. Yapıcı, isteğe bağlı bir zaman damgası parametresi kabul etmekte; atlanması durumunda std::strftime kullanılarak mevcut sistem saati biçimlendirilip atanmaktadır. toString() yardımcı yöntemi, tanılama çıktısı için yapıyı insan tarafından okunabilir bir günlük satırına dönüştürmektedir.

B. Alert Yapısı ve Şiddet Enum'u

Tespit edilen tehditler, tetiklenen kuralın adını (ruleName), dört düzey şiddet sınıflandırmasını (Severity::LOW, MEDIUM, HIGH, CRITICAL), insan tarafından okunabilir bir açıklamayı ve tetikleyici APICall'a sahiplik içermeyen ham bir işaretçiyi kayıt altına alan Alert yapısıyla temsil edilmektedir. Konsol çıktısı için ANSI renk kodlarına eşleme yapılmaktadır: düşük için yeşil, orta için sarı, yüksek için kırmızı ve kritik için macenta.

IV. SINIF HİYERARŞİSİ

A. DetectionRule — Soyut Taban Sınıf

DetectionRule, tüm tespit kurallarının uygulamak zorunda olduğu arayüzü tanımlamaktadır. Tek saf sanal yöntemi olan checkBehavior(), şimdiye kadar biriktirilmiş çağrı geçmişini ve değerlendirilecek mevcut APICall'u kabul ederek (boş olabilecek) bir Alert vektörü döndürmektedir. Sanal yıkıcı, türev nesnelerin taban sınıf işaretçileri aracılığıyla doğru biçimde yok edilmesini sağlamaktadır. countAPI() ve argContains() olmak üzere iki korumalı yardımcı yöntem, alt sınıflar arasında kod tekrarını önleyen yeniden kullanılabilir sorgu temel işlevleri sağlamaktadır.

TABLO I. Sınıf Hiyerarşisi ve NYP İlkeleri

Sınıf	Tür	Gösterilen NYP İlkeleri
DetectionRule	Soyut taban sınıf	Soyutlama (saf sanal arayüz), Kapsülleme (korumalı üyeler)
RansomwareRule	Somut türev sınıf	Kalıtım, Polimorfizm (checkBehavior override)
CodeInjectionRule	Somut türev sınıf	Kalıtım, Polimorfizm (checkBehavior override)
SuspiciousNetworkRule	Somut türev sınıf	Kalıtım, Polimorfizm (checkBehavior override)
DetectionEngine	Düzenleme sınıfı	Kapsülleme (özel rules_ / history_), Polimorfizm (sanal yönlendirme)
APICall	Düz veri yapısı	Kapsülleme (veri birleştirme)
Alert	Düz veri yapısı	Kapsülleme (sonuç birleştirme)

B. RansomwareRule

RansomwareRule üç tespit alt kuralı uygulamaktadır. Birinci alt kural, bağımsız değişken listesinde 'vssadmin', 'wmic' veya 'shadowcopy' dizelerini içeren CreateProcess veya ShellExecute çağrısında tetiklenerek Gölge Kopya silme girişimine işaret etmektedir ve CRITICAL şiddet atar. İkinci alt kural, şifreleme sonekli hedeflere (.encrypted, .locked, .cry, .ransom) yönelik MoveFile veya CopyFile çağrılarını tespit ederek HIGH şiddet atar. Üçüncü alt kural, geçmişte aynı sürecin WriteFile çağrılarını sayar; yapılandırılabilir writeThreshold aşıldığında HIGH şiddetli toplu şifreleme uyarısı verir.

C. CodeInjectionRule

CodeInjectionRule, kurallasalmış üç adımlı DLL/kabuk kodu enjeksiyon dizisini tespit etmektedir. Daha önce OpenProcess çağırılmış bir süreçten VirtualAllocEx gözlemlendiğinde uzak bellek tahsisini gösteren HIGH şiddetli uyarı verilir. Geçmişinde hem OpenProcess hem de VirtualAllocEx bulunan bir süreçten WriteProcessMemory gözlemlendiğinde enjeksiyon üçlüsünün tamamlandığını doğrulayan CRITICAL uyarı verilir. CreateRemoteThread, önceki API çağrılarından bağımsız olarak her zaman CRITICAL uyarı tetikler.

D. SuspiciousNetworkRule

SuspiciousNetworkRule veri sızıntısı senaryolarını ele almaktadır. Connect ve WSASend çağrılarını izleyerek aynı süreçten önceden ReadFile gözlemlendiğinde MEDIUM şiddetli sızıntı uyarısı verir. Bunun yanı sıra 4444, 1337 ve 31337 portlarına bağlantı girişimleri — bilinen Metasploit ve arka kapı portları — önceki dosya okumalarından bağımsız olarak HIGH şiddetli K2 iletişimi uyarısı tetikler.

V. TESPİT MOTORU

DetectionEngine, merkezi düzenleme sınıfıdır. İki özel üye tutmaktadır: kayıtlı tüm kural nesnelerinin sahipliğini elinde bulunduran unique_ptr<DetectionRule>'dan oluşan rules_ vektörü ve şimdiye kadar işlenen olayların akışını temsil eden history_ vektörü.

processCall() yöntemi, temel yönlendirme döngüsünü uygulamaktadır. Kayıtlı her kural için taban sınıf işaretçisi aracılığıyla rule->checkBehavior(history_, call) çağrısını gerçekleştirir. checkBehavior() sanal olduğu için C++ çalışma zamanı, motorun somut türü bilmesine gerek kalmaksızın doğru somut override'a yönlendirmektedir. Bu, polimorfizm ilkesinin bizzat uygulamasıdır: motor değişikliğe kapalı, genişlemeye açıktır — Açık/Kapalı İlkesinin doğrudan gerçekleştirilmesi [10].

reset() yöntemi, rules_'ı etkilemeksizin history_'yi temizleyerek aynı motor örneğinin birbirinden bağımsız birden fazla senaryoyu sırayla işlemesine olanak tanımaktadır.

VI. SİMÜLASYON SENARYOLARI VE SONUÇLAR

A. Senaryo 1: Fidyeye Yazılımı (cryptolocker.exe)

Sekiz API çağrısı bir fidye yazılımı yürütme zincirini simüle etmektedir. İlk çağrı bir belge açmakta, ikincisi onu okumakta ve sonraki dört WriteFile çağrısı .encrypted sonekliyle şifreli sürümleri yazmaktadır. Dördüncü WriteFile ile toplu şifreleme eşiği (4 olarak belirlenmiş) aşılarak HIGH uyarısı tetiklenir. .encrypted hedefine yönelik bir MoveFile çağrısı daha sonra HIGH dosya yeniden adlandırma

uyarısı tetikler. Son olarak 'vssadmin delete shadows' içeren bir CreateProcess çağrısı CRITICAL Gölge Kopya silme uyarısı oluşturur.

B. Senaryo 2: Süreç Enjeksiyonu (injector.exe → svchost.exe)

Dört API çağrısı klasik OpenProcess → VirtualAllocEx → WriteProcessMemory → CreateRemoteThread enjeksiyon dizisini yeniden oluşturmaktadır. OpenProcess'in ardından hiçbir uyarı tetiklenmez. VirtualAllocEx, geçmişte OpenProcess zaten mevcut olduğu için HIGH uyarısı tetikler. WriteProcessMemory, enjeksiyon üçlüsünün üçüncü üyesi olarak CRITICAL uyarısı tetikler. CreateRemoteThread ise uzak iş parçacığı oluşturma için ikinci bir CRITICAL uyarısı tetikler.

C. Senaryo 3: Veri Sızıntısı (spyware.exe)

spyware.exe iki hassas dosyayı okumakta ve ardından 192.168.100.55:4444 adresine bir TCP bağlantısı başlatmaktadır. Connect çağrısı iki eş anlı uyarı tetikler: ReadFile önceden eşlik ettiği için MEDIUM sızıntı uyarısı ve port 4444 bilinen-kötü listesinde olduğundan HIGH K2 port uyarısı.

D. Senaryo 4: Temiz Uygulama (notepad.exe)

notepad.exe tek bir metin dosyası üzerinde dört işlem gerçekleştirmektedir: OpenFile, ReadFile, WriteFile ve CloseFile. Bu çağrılarının hiçbirisi hiçbir kural koşulunu karşılamaz; hiçbir uyarı üretilmez. Bu durum, tespit mantığının zararsız tek dosya düzenleme davranışı için yanlış pozitif üretmediğini doğrulamaktadır.

TABLO II. Senaryo Özeti ve Uyarı Sonuçları

Senaryo	Süreç	API Çağrısı	Üretilen Uyarılar
1 — Fidyeye Yazılımı	cryptolocker.exe	8	HIGH (toplu yazma ×3, yeniden adlandırma), CRITICAL (gölge kopya silme)
2 — Enjeksiyon	injector.exe	4	HIGH (VirtualAllocEx), CRITICAL (WriteProcessMemory, CreateRemoteThread)
3 — Sızıntı	spyware.exe	4	MEDIUM (sızıntı ×2), HIGH (K2 port ×2)
4 — Temiz	notepad.exe	4	Yok (doğru negatif doğrulandı)

VII. TARTIŞMA

A. NYP Tasarım Değerlendirmesi

Dört NYP ilkesi bu projede somut ve önemsiz olmayan biçimde gerçekleştirilmektedir. Soyutlama, bir tespit kuralının ne yapması gerektiğini (davranışı değerlendir ve uyarı döndür) nasıl yapılacağını belirtmeksizin tanımlayan DetectionRule ile gösterilmektedir. Kalıtım, taban sınıftan countAPI() ve argContains() yardımcıları yeniden kullanırken tamamen farklı checkBehavior() uygulamaları sağlayan üç türev kural sınıfı ile gösterilmektedir. Polimorfizm, sıfır motor tarafı koşul ile taban sınıf işaretçisi aracılığıyla doğru kural uygulamasını çağıran DetectionEngine::processCall() ile gösterilmektedir.

Kapsülleme ise yalnızca `addRule()`, `processCall()` ve `reset()` arayüz yöntemleri aracılığıyla erişilebilen `DetectionEngine`'nin özel `rules_` ve `history_` üyeleri ile gösterilmektedir.

B. Genişletilebilirlik

Yeni bir tespit kuralı eklemek yalnızca şu adımları gerektirmektedir: (1) `DetectionRule`'dan yeni bir sınıf türetmek; (2) `checkBehavior()` uygulamak; (3) `engine.addRule()` ile bir örnek kaydetmek. `DetectionEngine` veya mevcut hiçbir kural sınıfında değişiklik yapılmasına gerek yoktur. Bu genişletilebilirlik, sanal yönlendirme mekanizması aracılığıyla gerçekleştirilen Açık/Kapalı İlkesinin doğrudan bir sonucudur.

C. Sınırlılıklar

API çağrı akışı yapaydır; bir üretim sistemi canlı API çağrılarını kesmek için çekirdek modu izleme sürücülerıyla entegre olmak zorunda kalacaktır. Tespit kuralları basit sayaç eşikleri ve dize eşleştirmesi kullanmaktadır; saldırganlar `WriteFile` çağrılarını daha uzun zaman pencerelerine yayarak bunlardan kolayca kaçabilir. `history_` vektörü sınırsızdır ve işlenen çağrı sayısı ile doğrusal olarak büyümektedir; uzun süreli dağıtım için yapılandırılabilir saklama süresi olan kayan zaman penceresi gerekecektir.

D. Üretim Sistemleriyle Karşılaştırma

Bu aracın mimarisi, aynı zamanda gelen paketleri takılabilir tespit kuralları kümesine karşı değerlendiren Snort ve Suricata ağ saldırı tespit sistemlerindeki kural motoruyla yapısal benzerlikler taşımaktadır [11]. Temel farklar ölçek, kural dili ve dağıtım bağlamıdır. Bu aracın pedagojik değeri, bu tür sistemlerin kavramsal işleyişini alana özgü dil ek yükü olmaksızın doğrudan okunabilir ve değiştirilebilir kılmasındadır.

VIII. SONUÇ

Bu çalışmada, nesneye yönelik programlamanın dört temel direğini açıkça gösteren C++17'de uygulanmış davranışsal bir API çağrı izleme ve tehdit tespiti sistemi sunulmuştur. Soyut `DetectionRule` taban sınıfı, sonsuz sayıda davranışsal kural için istikrarlı bir arayüz sağlamaktadır; üç somut alt sınıf — fidyeye yazılımı, kod enjeksiyonu ve ağ sızıntısını kapsayan — paylaşılan yardımcı yöntemleri yeniden kullanırken farklı tespit mantıklarını uygulamaktadır; `DetectionEngine` tüm kuralları sanal yönlendirme aracılığıyla polimorfik olarak uygulamaktadır; tüm kod tabanında özel üye erişimi temiz kapsülleme sınırlarını uygulamaktadır. Dört simülasyon senaryosu bilinen kötü niyetli API kalıplarının doğru tespitini doğrulamakta ve zararsız iş yüklerinde yanlış pozitif üretilmediğini onaylamaktadır.

TEŞEKKÜR

Yazarlar yapıcı geri bildirimlerinden dolayı hakeme teşekkür eder. Bu çalışma bir Nesneye Yönelik Programlama dersi proje ödevi kapsamında yürütülmüş olup herhangi bir dış fon desteği almamıştır.

KAYNAKLAR

- [1] M. Sikorski ve A. Honig, Pratik Kötü Amaçlı Yazılım Analizi. San Francisco, CA: No Starch Press, 2012.
- [2] FireEye Mandiant, "M-Trends 2023: Özel Rapor," FireEye, Inc., 2023. [Çevrimiçi]. Erişim: <https://www.mandiant.com/m-trends>

- [3] C. Willems, T. Holz ve F. Freiling, "CWSandbox kullanılarak otomatik dinamik kötü amaçlı yazılım analizine doğru," IEEE Secur. Priv., cilt 5, sayı 2, ss. 32–39, 2007.
- [4] C. Guarnieri ve ark., "Cuckoo sanal alanı: Otomatik kötü amaçlı yazılım analizi," in USENIX LEET, Washington, DC, 2013.
- [5] U. Bayer ve ark., "TTAnalyze: Kötü amaçlı yazılım analizi aracı," in EICAR Konf., Hamburg, Almanya, 2006.
- [6] K. Rieck ve ark., "Makine öğrenimi kullanılarak kötü amaçlı yazılım davranışının otomatik analizi," J. Comput. Secur., cilt 19, sayı 4, ss. 639–668, 2011.
- [7] Z. Fang ve ark., "MalBERT: Çift yönlü kodlayıcı gösterimleri kullanılarak kötü amaçlı yazılım tespiti," Comput. Secur., cilt 128, s. 103176, 2023.
- [8] E. Gamma, R. Helm, R. Johnson ve J. Vlissides, Tasarım Kalıpları. Reading, MA: Addison-Wesley, 1994.
- [9] J. Richter ve C. Nasarre, Windows via C/C++, 5. baskı. Redmond, WA: Microsoft Press, 2008.
- [10] R. C. Martin, Temiz Mimari. Upper Saddle River, NJ: Prentice Hall, 2017.
- [11] M. Roesch, "Snort: Ağlar için hafif saldırı tespiti," in USENIX Sist. Yön. Konf., Seattle, WA, 1999, ss. 229–238.